

QA Discovery-Channel Ledger

Revision: 3 **Last modified:** 2026-08-10T17:30:55Z **Status:** active

Constitution: Â§11.4.238 (automated QA must be the DISCOVERER, not the confirmer “ every defect found outside the automated HelixQA regime is itself a coverage-escape release blocker, not merely a bug to fix).

Purpose

Every defect this project closes MUST record **which channel found it:** automated-helixqa (the standing HelixQA regime caught it) or an **out-of-band channel** “ manual-qa, operator-report, agent-code-reading, or incidental-discovery. Per Â§11.4.238(C), every out-of-band entry MUST carry a **coverage-escape audit:** why the automated regime missed it (cited to the specific missing/blind check, never “œwe didnâ€™t think of itâ€”), and the new or strengthened automated check “ with its own Â§11.4.115 RED capturing the escaped defect “ that closes the gap.

Per Â§11.4.238(E), this project tracks the discovery-channel split over time and drives the out-of-band share toward zero. **Honest boundary (Â§11.4.6):** this ledger starts wherever this projectâ€™s actual history is “ see the retroactive seed entries below, all found via agent-code-reading during the 2026-08-07/2026-08-08 governance audits, exactly the channel this anchor targets. Full retroactive audit of every historical closure in docs/Fixed.md (~62 items) is NOT attempted here “ that is out of proportion for a ledger just being stood up, and would itself be a Â§11.4.6 fabrication if backfilled without real investigation per entry. The ledger is honest about starting now, not claiming a false complete history.

Schema (per entry)

Field	Meaning
id	The workable item id (BOB-NNN) if one exists, else a short slug
date	ISO date the defect was found
channel	automated-helixqa manual-qa operator-report agent-code-reading incidental-discovery
summary	One line: what was wrong
escape-audit	(out-of-band only) which check was missing/blind, cited by name/path
new-check	(out-of-band only) the new/strengthened check that now catches this class, with its RED-capture evidence

Entries

RD2-22 “PATCH /schedules/{id} and PUT /theme unauthenticated with a token set

- **id:** RD2-22 (governance-audit item, not yet a BOB-NNN)
- **date:** 2026-08-07 (found by the 2026-08-07 governance audit), confirmed still open + fixed 2026-08-08
- **channel:** agent-code-reading “ found by direct source inspection (comparing sibling routes’™ Depends(require_api_token) presence), not by any test or HelixQA run.
- **escape-audit:** tests/security/test_hooks_schedules_auth.py’™s _MUTATING enumeration (the ONLY automated check covering this auth surface) was scoped to exactly 4 routes (POST/DELETE hooks, POST/DELETE schedules) when the original RW-02 fix landed “ it was never extended when PATCH /schedules/{id} and PUT /theme were added as separate routes. The check existed, ran green, and was simply never told about two of the six routes it should have covered “ a scope gap, not a broken check (per the ledger’™s channel split, this is the “œgenuinely uncovered” class, not “œexisted-but-missed”).
- **new-check:** _MUTATING extended to all 6 routes (tests/security/test_hooks_schedules_auth.py), RED-captured (confirmed 4/4 new-route assertions failed with expected 401, got 200 before the fix), now GREEN (31/31). See docs/GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-22 for the full before/after evidence.

RD2-41a “ scripts/docs_chain.sh Step 1/3 silently no-op” d on every run

- **id:** RD2-41 (governance-audit item)
- **date:** 2026-08-08
- **channel:** agent-code-reading “ found by directly invoking docs_chain.sh and reading its own printed error line, then tracing the hardcoded path in source.
- **escape-audit:** **no automated check existed at all** for “œdoes docs_chain.sh’™s Step 1 actually run the real DB export” “ the script printed an ERROR: line on every failure but nothing consumed/asserted on that exit status anywhere in the test suite or pre-build gate; a human had to actually run it and read the output. Genuinely uncovered, not a broken check.
- **new-check:** tests/unit/test_docs_chain_binary_resolution.sh “ real-invocation test asserting Step 1 resolves a real binary and reaches the real validate call. RED-captured (confirmed failing against the pre-fix script: FAIL: Step 1/3 still reports 'binary not found'), now GREEN.

RD2-41b “ scripts/pre_build_verification.sh invariant 17 silently SKIPPED on every run

- **id:** RD2-41 (governance-audit item, same root cause as RD2-41a)
- **date:** 2026-08-08
- **channel:** agent-code-reading “ found while fixing RD2-41a, by checking for the identical bug pattern in this sibling script per the project’s own extend-to-all-cases discipline.
- **escape-audit:** this is the sharpest instance in this ledger of “the checker itself was blind”: invariant 17 IS the pre-build gate’s own workable-items DB-integrity check, and it was silently SKIPPING (not failing “ the script’s own else branch treats a missing binary as an honest skip, not a failure) on every single pre-build run since the path was wrong. A gate whose own precondition-check is broken cannot report the truth about the thing it gates “ an existed-but-missed class defect (the check existed, ran, and was blind), the more dangerous of the two classes per this ledger’s schema.
- **new-check:** tests/unit/test_pre_build_workable_items_invariant.sh “ real-invocation test asserting invariant 17 reaches a real PASS/FAIL verdict instead of silently skipping. RED-captured (confirmed failing against the pre-fix script via git stash/restore), now GREEN. **Residual, still-open finding from this same investigation** (tracked as RD2-41’s own note in docs/GOVERNANCE_AUDIT_2026-08-08_ROUND2.md, not yet closed): a SEPARATE, earlier mutation-marker carrier false-positive in the same script aborts the ENTIRE pre-build gate before invariant 17 is ever reached in a full run “ meaning this fix, while correct, is not yet provably exercised end-to-end in production; the ledger records this honestly rather than claiming full closure.

BOB-008 “ DB’s ĨZMD body drift + missing enumerated unblock choices

- **id:** BOB-008
- **date:** 2026-08-08
- **channel:** agent-code-reading “ found by running workable-items diff/validate during a routine sync check, not by any standing automated gate (no CI job runs workable-items diff on a schedule).
- **escape-audit:** no automated check runs workable-items diff/validate as a standing, scheduled, or write-seam-triggered gate “ Â§11.4.106(F)’s commit-seam sync hook (which would have caught the DB write in commit 54e313f landing with no matching docs/Issues.md update) does not exist in this repo. Genuinely uncovered.
- **new-check: closed 2026-08-08.** scripts/pre_build_verification.sh invariant 17 extended to run workable-items diff (DB-vs-Markdown divergence) alongside validate (internal DB invariants) “ it now catches BOTH the Â§11.4.148(D3) enumerated-choices class AND the DB’s ĨZMD body-drift class. RED-captured via tests/unit/test_pre_build_workable_items_diff_check.sh’s Â§1.1 paired mutation (a deliberately desynced docs/Issues.md “ confirmed FAIL

with the diff check absent, confirmed correct divergence-specific FAIL, distinguished from the separate pre-existing BOB-009/010 validate issue so the test does not tautologically reuse an unrelated failure “ then GREEN after the fix, with a companion assertion that the real synced tree reports no false divergence).

RD2-42 “ podman ps reports a container “Up “ (healthy)“ while its process is genuinely dead

- **id:** RD2-42 (governance-audit item)
- **date:** 2026-08-09
- **channel:** incidental-discovery “ surfaced while attempting the RD2-22 live curl-verify: curl http://localhost:7187/health returned HTTP=000 (connection refused), while podman ps simultaneously reported qbittorrent-proxy: Up 15 hours (healthy).podman exec qbittorrent-proxy ... returned OCI runtime error: crun: ... is not running and podman inspectNetworkSettings.Ports was empty {} “ the container’s actual crun process was dead (no restart count, no OOMKilled flag recorded “ podman’s own state was simply stale/desynced from reality), most likely fallout from the same host session-kill mechanism already tracked in docs/incidents/ reaching into the rootless-podman container process tree.
- **escape-audit: no automated check exists at all** for “is podman ps’s reported state consistent with the container’s real crun/process state“ “ this is exactly the Â§11.4.196(F)/Â§11.4.201(6) “configured “ in use“ false-null class applied to container orchestration: a healthy-looking podman ps line is not proof the service is reachable. Genuinely uncovered “ no pre-build/pre-test gate probes the live stack’s actual reachability before a test run assumes it.
- **new-check:** not yet authored (tracked, not yet closed). Candidate: a tests/fixtures/services.py-level or pre-build-gate-level real-reachability probe (curl/socket-connect to each mapped port, not podman ps text) run before any live-stack- dependent test session, with an honest recreate-and-retry path “ mirroring what this session did manually (./start.sh --recreate, confirmed HTTP=200 after ~90s).
- **resolution applied this session:** ./start.sh --recreate (the project’s sanctioned orchestrator, never raw podman restart) “ confirmed curl :7187/health “ 200 after the stack’s normal ~90s startup window.

RD2-43 “ bare python3 -m pytest silently fails ALL collection via a stale ~/.local rpbs build

- **id:** RD2-43 (governance-audit item)
- **date:** 2026-08-09
- **channel:** incidental-discovery “ surfaced when relaunching the interrupted `tests/contract/
 - tests/unit/regression sweep with a barepython3 -m pytest invocation: immediateModuleNotFoundError: No module

- named `~rpds.rpds`™ at collection time (inside `theschemathesis`pytest-plugin`™'s import chain), aborting the ENTIRE sweep before a single test ran.
- **escape-audit:** host-level Python version drift “ the system `python3` resolves to Python 3.14.6, but `~/local/lib/python3/site-packages/rpds/rpds.cpython-313-x86_64-linux-gnu.so` is a native extension built for the 3.13 ABI (real ABI mismatch, confirmed via `ls ../rpds/*.so + python3 --version`). The project™'s own `.venv` (Python 3.14.6, `.venv/bin/python3 -c "import rpds"` succeeds) is the correct, working interpreter “ no automated check enforces “œœtests are always run via `.venv/bin/python3`, never a bare `python3`“ anywhere in this repo™'s own tooling (no Makefile/wrapper script that fails closed on the wrong interpreter). Genuinely uncovered; this is an environment-fragility class distinct from the source-level bugs this ledger otherwise tracks, but it silently produces a 100%-collection-failure that could be misread as “œœevery test in the suite is broken“ by anyone (human or agent) who doesn™'t already know to check the interpreter.
 - **new-check:** not yet authored (tracked, not yet closed). Candidate: a thin `scripts/run_tests.sh` wrapper (or a `pyproject.toml`/CI-adjacent guard) that verifies `sys.prefix` resolves inside `.venv/` before invoking `pytest`, failing closed with an actionable message rather than a confusing plugin-internals traceback.
 - **resolution applied this session:** relaunched the sweep via `.venv/bin/python3 -m pytest` “ confirmed genuinely running (real per-test PASS lines, not a collection error).

RD2-44 “ test_get_existing_search_returns_200”™s 120s poll window had near-zero real margin

- **id:** RD2-44 (governance-audit item, same root-cause FAMILY as the already-fixed `tests/e2e/test_full_pipeline.py` timeouts, but a genuinely separate finding/fix “ that fix did not touch this file)
- **date:** 2026-08-09
- **channel:** incidental-discovery “ surfaced re-running `tests/integration/test_merge_api.py` (RD2-26a™'s own deliverable) against the actually-live stack for the first time since its original mocked-service replacement; it had only ever been run with the stack unreachable (28 passed / 20 skipped) until this session.
- **escape-audit:** RD2-26a™'s own author (a parallel subagent, same session) had no way to discover this “ the file was authored and its author™'s own live-verification attempt happened while the stack was down, so it never actually exercised this code path against a live, contended host before committing. No automated check re-runs the live-service test suite on a schedule independent of whether an agent happens to have the stack up at authoring time “ genuinely uncovered.
- **new-check:** the test itself IS the check; it was simply mistimed. Live-measured real search completion (42 trackers, real network calls, under real concurrent 4-subagent host load): ~118s once, ~298s under heavier concurrent load “ both within a few seconds of, or past, the

old 120s ceiling. Widened to 300s in tests/integration/test_merge_api.py, matching the test_full_pipeline.py pattern's order of magnitude. RED-adjacent evidence: the original 120s-deadline run genuinely failed (assert 'running' != 'running') against the live stack; post-fix re-run: 1 passed in 297.72s.

INC-2026-08-10 " GCM auth-bypass mutation left in qBitTorrent-go/internal/db/crypto.go during Agent H's RED window

- **id:** INC-2026-08-10 (this-session incident; not a governance-audit item " inline coverage-escape captured during autonomous work by the conductor)
- **date:** 2026-08-10
- **channel:** automated_background_scan " a background security-review plugin (out-of-cycle from the pre-build regime) caught the mutation during Agent H's Case (c) RED-then-GREEN window (~few seconds) while it was authoring RD2-31 chaos tests in qBitTorrent-go/tests/integration/jackett_db_test.go.
- **escape-audit:** subagent briefing template for RD2-31 said "Do NOT touch production code paths " ONLY the test file" but was PROSE, not mechanically enforced. Agent H interpreted narrowly and mutated qBitTorrent-go/internal/db/crypto.go Decrypt (if err != nil { if false && err != nil, swallowing GCM authentication failures) for its RED assertion, then restored to byte-identical HEAD. The window was small but real " a \$11.4.84 working-tree-quiescence violation. No boba-side pre-tool-use hook would have caught the mutation during the window (only a post-fact scan would); the out-of-cycle security scan caught it BEFORE restoration. Genuinely uncovered by the in-cycle regime.
- **new-check:** \$11.4.84 fence LANDED "
 - **Layer 1** (subagent briefing template with three mandatory isolation patterns: git worktree per \$11.4.179, atomic mutate-run-restore Bash tool call with trap, mutation-testing tool operating on isolated copy) APPLIED to every subsequent subagent dispatch (proven: Agents K + L both used the strengthened briefing without violation).
 - **Layer 3** (scripts/pre_build_verification.sh invariant 23 CM-NO-PRODUCTION-MUTATION-RESIDUE, with FIXTURE_ROOT env-testable golden-good/golden-bad fixtures under scratchpad/agent-L-fixtures/) LANDED " real-repo PASS today (0 residue in production paths).
 - **Layer 2** (post-tool-use hook enforcing mid-window detection) DEFERRED pending Claude Code runtime-capability verification (currently PreToolUse hooks only per \$11.4.109 precedent).
 - Design doc: scratchpad/task-20-84-fence-design.md.
- **resolution applied this session:** Agent H's crypto.go restoration was clean (verified via git status empty + git diff empty + full-tree grep for MUTATED patterns in production paths = empty); no commit contamination (main stream was holding for atomic commit per

Â§11.4.121). Â§11.4.84 fence Layer 1 + Layer 3 both landed and verified. Layer 2 tracked as Â§11.4.197 follow-up.

Discovery-channel split (tracked, per Â§11.4.238(E))

Period	automated-helixqa	out-of-band (all channels)	out-of-band %
2026-08-07 â†’ 2026-08-10 (ledger current)	0	8 (agent-code-reading x4, incidental-discovery x3, automated_background_scan x1)	100%

Honest note: 100% out-of-band is the true, unflattering starting number â€” every entry above was found by an agent reading code, running commands by hand, or hitting a real failure while doing unrelated live-verification work; none by a standing HelixQA run. This is exactly the state Â§11.4.238 exists to change; the target is this percentage trending toward zero as the new-check column above closes each specific gap (RD2-42/ RD2-43 are still open â€” no automated check authored yet, tracked honestly as such) and as future work is driven through the HelixQA banks (challenges/helixqa-banks/) rather than ad-hoc investigation.